



GATE CS / IT 2027

Compiler Design

Quick Revision Sheet

Complete GATE syllabus coverage — All topics and subtopics

Compilation Phases

All 6 Phases

- 1. Lexical Analysis: source $\rightarrow$ token stream
- 2. Syntax Analysis (Parser): tokens $\rightarrow$ parse tree
- 3. Semantic Analysis: type checking, scope resolution
- 4. Intermediate Code Generation: 3-address code
- 5. Code Optimization: improve IC
- 6. Code Generation: IC $\rightarrow$ target machine code
- Symbol Table used throughout all phases

Parsing

Top-Down (LL)

- LL(1): Left-to-right scan, Leftmost derivation, 1 lookahead
- FIRST(X): terminals that can start strings derived from X
- FOLLOW(X): terminals that can appear immediately after X
- LL(1) table: no conflicts = grammar is LL(1)
- Recursive descent: simple top-down parser

Bottom-Up (LR)

- $LR(0) < SLR(1) < LALR(1) < CLR(1)$ in power
- LALR(1) used in YACC/Bison - most practical
- LR parsing: shift and reduce operations
- Conflicts: shift-reduce and reduce-reduce
- LR parsers handle all unambiguous CFGs

Syntax-Directed Translation

Attributes & Code

- Synthesized attribute: value from children
- Inherited attribute: value from parent or siblings
- S-attributed SDD: only synthesized (works with LR)
- L-attributed SDD: synthesized + left-inherited (works with LL)
- Three-address code: $x = y \text{ op } z$
- Quadruple: (op, arg1, arg2, result)

Code Optimization

Techniques

- Constant folding: compute constant expressions at compile time
- Dead code elimination: remove unreachable/unused code
- CSE (Common Subexpression Elimination): reuse computed values
- Loop invariant code motion: hoist loop-independent code
- Strength reduction: $x*2 \rightarrow x+x$, $x*4 \rightarrow x<<2$
- Inlining: replace function call with function body